SignBridge PK

Team Work Distribution & Git Commit Guidelines

Real-time Pakistan Sign Language Alphabet Recognizer (Track A)

Item	Decision
GitHub repository name	**signbridge-pk** (lowercase, hyphenated)
Team	Aliza (Data lead), Fatima (Model lead), Anas (App & DevOps lead)
Track	A: Application Development (web app with real-time prediction; deployment excluded, app runs locally)
Stack	Python 3.11, PyTorch + torchvision, scikit-learn, MediaPipe, OpenCV, Gradio, pytest, ruff
Report	LaTeX only (proposal and final technical report), compiled PDF submitted with the repo URL

Check the name is free on GitHub before creating it. If taken, use **signbridge-psl**. Anas creates the repo and adds Aliza and Fatima as collaborators on day one.

1. Roles at a glance

Member	Role	Leads phases	Owns (folders)
Aliza	Data lead	Phase 2 (Data), bias check in Phase 4	src/signbridge/data/, data/ (docs only), scripts/collect_*, docs/dataset_card.md
Fatima	Model lead	Phase 3 (Implementation), Phase 4 (Evaluation)	src/signbridge/features/, models/, evaluation/, configs/, notebooks/, docs/model_card.md
Anas	App & DevOps lead	Phase 5 (Output), repo setup, Phase 6 packaging	app/, src/signbridge/inference/, tests/, .github/, README.md

Everyone shares Phase 1 (problem definition) and Phase 6 (LaTeX report). Names can be swapped between roles if your team prefers; keep the structure the same.

2. Deliverable plan with dates

Your instructions say the project is submitted in several deliverables, each covering 2 or 3 phases, and the exact grouping and deadlines are announced before each cycle. This plan uses the example grouping from the instructions (Deliverable 1 = Problem Definition + Data Acquisition) and **proposed internal deadlines**, set earlier than any official date because no late submissions are accepted and you must submit at least 24 hours early. Fill in the official dates when your instructor announces them and shift the plan to match. Deployment is excluded (confirmed by your TA), so the app runs locally.

Deliverable	Phases covered	Proposed work window	Internal deadline	Official deadline (fill in)
Setup	Topic approval, repo	Mon Oct 5 - Fri Oct 9	Fri Oct 9	-
Deliverable 1	1 Problem Definition, 2 Data Acquisition & Setup	Mon Oct 5 - Fri Oct 23	Fri Oct 23	_____
Deliverable 2	3 Implementation & Execution, 4 Evaluation & Comparative Analysis	Mon Oct 26 - Fri Nov 20	Fri Nov 20	_____
Deliverable 3	5 Output (web app), 6 Documentation & Final Report	Mon Nov 23 - Fri Dec 11	Fri Dec 11	_____

Deliverable	Phases covered	Proposed work window	Internal deadline	Official deadline (fill in)
Buffer	Fixes, rehearsal, final check	Mon Dec 14 - Fri Dec 18	-	-

The work windows assume about 11 weeks from today (Oct 4, 2026). If the real deadlines are closer, drop the optional items (small CNN baseline, build-a-word) first.

Setup: topic approval and repository (Oct 5 - Oct 9)

Task	Owner	Dates	Output
Enter the project summary in the shared Google sheet and wait for the status to say Approved before any other work	Anas (all review)	Oct 5 - Oct 6	Sheet row marked Approved
Create repo signbridge-pk, add collaborators, protect main, add scaffold, .gitignore, requirements, CI (ruff + pytest)	Anas	Oct 5 - Oct 9	Repo live, CI passing
Each member clones, sets git identity, makes a first commit on a branch and opens a pull request	All	Oct 7 - Oct 9	3 merged PRs, one per member

Deliverable 1: Phases 1 and 2 (Oct 5 - Oct 23)

Task	Owner	Dates	Output
Problem statement, PSL scope (static letters only), list of public PSL/ASL datasets, sign list checked by a PSL source or user	Aliza	Oct 5 - Oct 12	docs/scope.md, proposal section
Candidate models, evaluation metrics and success targets	Fatima	Oct 5 - Oct 12	Proposal section
User stories and UI wireframe	Anas	Oct 9 - Oct 14	Wireframe in proposal
Compile the LaTeX proposal and proofread together	All (Anas assembles)	Oct 13 - Oct 16	docs/proposal/proposal.pdf
Data collection protocol and consent note; start collecting volunteer images (5-10 people)	Aliza	Oct 8 - Oct 21	Raw images (kept out of Git)
Tiny fake manifest and dummy model so the app can be built early	Aliza + Fatima	Oct 8 - Oct 14	Dummy files committed
MediaPipe landmark extraction, normalisation and dataloader prototype	Fatima	Oct 12 - Oct 22	src/signbridge/features/
Clean and label data; build manifest.csv; person-wise split; leakage test	Aliza	Oct 19 - Oct 23	manifest.csv, tests/test_split.py
Data download script, dataset link in data/README.md, config loader	Anas	Oct 14 - Oct 21	scripts/download_data.py
Draft report sections: Introduction, Related Work, Dataset (Aliza); Methodology (Fatima); System Architecture (Anas)	Each member	Oct 19 - Oct 22	docs/report/ drafts
Internal review, mock questions, submit Deliverable 1	All	Oct 22 - Oct 23	Submission on GCR

Deliverable 2: Phases 3 and 4 (Oct 26 - Nov 20)

Task	Owner	Dates	Output
Augmentation pipeline, ASL baseline data prep, final person-wise splits	Aliza	Oct 26 - Nov 4	src/signbridge/data/augment.py
Train landmark classifiers (Random Forest, SVM, small MLP) with fixed seeds	Fatima	Oct 26 - Nov 6	Checkpoints, metrics JSON
Inference wrapper with Predictor interface and unit tests	Anas	Oct 26 - Nov 6	src/signbridge/inference/
MobileNetV2 transfer learning (and optional small CNN baseline)	Fatima	Nov 2 - Nov 13	Checkpoints, metrics JSON
Gradio UI skeleton (webcam capture and upload) using the dummy model	Anas	Nov 2 - Nov 13	app/app.py runs locally
Bias check by person, skin tone group, lighting and background; error analysis	Aliza	Nov 5 - Nov 17	reports/bias/*.csv
Comparison table, confusion matrices, per-class F1, similar-letter analysis	Fatima	Nov 9 - Nov 17	reports/figures/, results table
Latency and memory check on a laptop CPU; tests that lock expected metrics	Anas	Nov 12 - Nov 18	tests/, short benchmark note
Report sections: Dataset & Preprocessing, Bias Analysis (Aliza); Methodology, Results (Fatima); System Architecture (Anas)	Each member	Nov 16 - Nov 19	docs/report/ updated
Internal review, mock viva, submit Deliverable 2	All	Nov 19 - Nov 20	Submission on GCR

Deliverable 3: Phases 5 and 6 (Nov 23 - Dec 11)

Task	Owner	Dates	Output
Landmark overlay function and top-3 output in the Predictor	Fatima	Nov 23 - Nov 27	Predictor updated
Connect the real model to the UI; top-3 display and landmark overlay	Anas	Nov 25 - Dec 1	Working local app
Demo sample images; user testing with volunteers; record feedback	Aliza	Nov 23 - Dec 3	reports/user_testing.md
Build-a-word feature, friendly error messages, privacy note and disclaimer	Anas	Nov 27 - Dec 4	Final UI
Fix issues found in testing	Anas + Fatima	Dec 2 - Dec 7	Bug-fix PRs
Dataset card (Aliza) and model card (Fatima)	Aliza, Fatima	Dec 1 - Dec 7	docs/dataset_card.md, docs/model_card.md
README with setup, run steps, data links, project structure, limitations; 60-90 second demo video	Anas	Dec 4 - Dec 9	README.md, demo video link
Final LaTeX report: assemble, add Challenges Faced and Conclusion, compile, proofread	All (Anas assembles)	Dec 1 - Dec 10	docs/report/report.pdf
Mock viva rounds and live-change drills	All	Dec 7 - Dec 10	Notes per module

Task	Owner	Dates	Output
Clean the repo, check commit balance, submit repo URL and PDF via GCR	Anas	Dec 10 - Dec 11	Submission on GCR

Every Friday: a 15-minute check that all three members have committed that week, all pull requests are reviewed, and nobody is blocked. Fix an uneven commit split early, not the night before a deadline.

3. Detailed tasks by project phase

The instructions say each deliverable covers 2 or 3 phases and the exact grouping is announced before each cycle. This table is organised by the six phases in the project description, so it works however the deliverables are grouped. **Bold** marks the phase lead.

Phase	Aliza (Data)	Fatima (Models)	Anas (App & DevOps)
1. Problem Definition	Write problem statement and PSL scope (static letters only). List public PSL/ASL datasets. Get the sign list verified by a PSL source or user.	Define candidate models, metrics (accuracy, per-class F1, confusion matrix) and success targets.	Write user stories and a UI wireframe. Decide how the app is run locally (one command). **Create repo, scaffold, .gitignore, CI.**
2. Data Acquisition & Setup	**Collect PSL images from 5-10 consenting volunteers. Clean, label, deduplicate. Build manifest.csv with person_id and split. Document the dataset.**	Landmark extraction pipeline (MediaPipe), PyTorch dataset and dataloaders, feature caching.	Data download script, dataset hosting link (Kaggle or cloud), README data section, config loader.
3. Implementation & Execution	Augmentation pipeline, ASL baseline data prep, person-wise train/val/test split, leakage checks.	**Train landmark + RF/SVM/MLP, MobileNetV2 transfer learning, optional small CNN. Tune, seed, save checkpoints and metrics JSON.**	Inference wrapper, model loading, unit tests for data and inference, linting in CI.
4. Evaluation & Comparative Analysis	**Bias check:** performance by person, skin tone group, lighting, background. Error analysis on hard samples.	**Comparison tables, confusion matrices, per-class F1, similar-letter analysis, ablations.**	Latency and memory check on a normal laptop CPU. Regression tests that lock expected metrics.
5. Output (web app)	Curate demo sample images. Run user testing with volunteers and record feedback.	Expose `predict(image)` returning top-3 with confidence. Landmark overlay function.	**Gradio UI (webcam capture + upload), top-3 display, build-a-word, error handling, one-command local run, demo video.**
6. Documentation & Final Report	LaTeX: Introduction, Related Work, Dataset & Preprocessing, Bias Analysis. Dataset card.	LaTeX: Methodology, model architecture diagrams, Results & Comparative Analysis. Model card.	LaTeX: System Architecture, Challenges Faced, Conclusion. Final README, repo cleanup, submission.

4. Individual task checklists

Aliza - Data lead

- Draft data collection protocol: letters list, hand positions, lighting, backgrounds, minimum images per letter per person.
- Write a consent note for volunteers; store no names, only anonymous person_id (p01, p02...).
- Collect and label images; remove blurry, duplicate and mislabeled shots.

- Build data/processed/manifest.csv with columns: path, label, person_id, split, lighting, background, skin_tone_group.
- Split **by person**, never randomly. Add a test that fails if one person_id appears in two splits.
- Implement augmentation (rotation, brightness, scale, flip only if the sign is mirror-safe) in src/signbridge/data/augment.py.
- Upload the dataset to Kaggle or cloud storage; add the link to the README; keep it out of Git.
- Run the bias analysis and write the dataset card.

Fatima - Model lead

- Implement MediaPipe landmark extraction and normalisation (translate to wrist, scale by hand size) so features ignore position and size.
- Train baseline classifiers on landmarks (Random Forest, SVM, small MLP) with fixed seeds.
- Fine-tune MobileNetV2 (freeze backbone first, then unfreeze last blocks). Optional small CNN baseline.
- Keep all hyperparameters in configs/*.yaml; save checkpoints to models/ and metrics to reports/metrics/*.json.
- Produce comparison table, confusion matrices, per-class F1, and a short note on confused letter pairs.
- Provide the single entry point `Predictor.predict(image) -> list of (label, probability)`.
- Write the model card and the Methodology and Results sections of the report.

Anas - App & DevOps lead

- Create the repo, branch protection, folder scaffold, .gitignore, requirements.txt, pre-commit (ruff) and CI (lint + tests).
- Build the Gradio app: webcam capture and upload, top-3 predictions, landmark overlay, build-a-word, clear messages when no hand is detected.
- Write inference tests with a mocked model so CI stays fast; add a smoke test that loads the real model.
- Make the app start with one command (python app/app.py) and test it on a clean clone on another machine.
- Write the README (setup, run, data links, project structure, limitations) and record the 60-90 second demo video.
- Write the System Architecture and Challenges sections; handle final repo cleanup and submission.

5. Interface contracts (so nobody blocks anybody)

Agree on these on day one and do not change them without telling the other two.

Handoff	Producer	Consumer	Contract
Dataset	Aliza	Fatima	data/processed/manifest.csv with columns path, label, person_id, split. Images are not in Git.
Landmark features	Fatima	Fatima, Anas	21 landmarks x (x, y, z) normalised to a 63-float vector in src/signbridge/features/landmarks.py.
Prediction	Fatima	Anas	`Predictor(model_path).predict(image: np.ndarray) -> list[tuple[str, float]]`, sorted by probability, top-3 by default. Raises `NoHandDetected` if no hand.
Bias results	Aliza	Fatima	reports/bias/*.csv with metric per subgroup.
Config	Fatima	All	configs/*.yaml; no hard-coded paths or magic numbers in code.

Until real data and models exist, Aliza and Fatima commit a tiny fake manifest and a dummy model so Anas can build and test the app without waiting.

6. Repository structure

```
signbridge-pk/
|-- README.md                  (Anas)
|-- requirements.txt           (Anas)
|-- pyproject.toml             (ruff, pytest config)
|-- .gitignore                 (datasets, checkpoints, venv, __pycache__)
|-- .github/workflows/ci.yml   (Anas: lint + tests)
|-- configs/                   (Fatima: yaml for training and paths)
|-- data/
|   |-- README.md              (Aliza: dataset links and how to download)
|   `-- processed/.gitkeep     (contents ignored by Git)
|-- scripts/                   (download, collect, train, evaluate)
|-- src/signbridge/
|   |-- data/                  (Aliza: manifest, split, augment, dataset)
|   |-- features/              (Fatima: landmark extraction)
|   |-- models/                (Fatima: classifiers, MobileNetV2 wrapper)
|   |-- evaluation/            (Fatima: metrics, plots; Aliza: bias)
|   `-- inference/             (Anas + Fatima: Predictor, errors)
|-- app/app.py                 (Anas: Gradio UI only, no ML logic)
|-- models/                    (checkpoints; small final model only)
|-- reports/                   (metrics json, figures, bias csv)
|-- notebooks/                 (exploration only, import from src)
|-- tests/                     (pytest for data, features, inference)
`-- docs/
    |-- proposal/              (LaTeX proposal)
    |-- report/                (LaTeX final report)
    |-- dataset_card.md        (Aliza)
    `-- model_card.md          (Fatima)
```

7. Git workflow and commit guidelines

Your instructions say commit history is analysed to verify individual contributions and uniform distribution is expected. These rules keep your history clean and fair.

Branches

- **main** is always working and deployable. Nobody pushes directly to main; use pull requests.
- Create short-lived branches named `type/owner-description`, for example `feat/aliza-person-wise-split`, `feat/fatima-mobilenet-training`, `fix/anas-no-hand-error`.
- Keep each branch to one task, merged within a few days.

Commit messages

Format: `type(scope): short imperative summary`. Types: feat, fix, data, exp, test, docs, refactor, ci, chore.

```
feat(data): add person-wise train/val/test split
fix(inference): return clear error when no hand is detected
exp(models): compare SVM and RF on normalised landmarks
test(data): fail if a person_id appears in two splits
docs(report): write dataset and preprocessing section
ci: run ruff and pytest on every pull request
```

Rules for fair, healthy history

- **Commit early and often:** target at least 3 meaningful commits per person per week. Each commit does one logical thing and the project still runs.
- **Aim for a roughly even split:** each member should land within about 25-40% of total commits. Do not pad with empty or trivial commits; reviewers can tell.
- **Each person commits their own work** from their own GitHub account. Never commit code under another person's name and never share a login.

- Set your identity: `git config user.name "Your Name"` and use the same email as your GitHub account so commits are attributed to you.
- **Pull requests:** at least one teammate review and passing CI before merging. Review rotation: Aliza reviews Anas, Anas reviews Fatima, Fatima reviews Aliza.
- Merge using **Rebase and merge** or a normal merge commit, not Squash, so individual commits stay visible in the history.
- Pull from main before starting work each day and resolve conflicts on your branch, not on main.
- Report-writing counts too: LaTeX section edits should be committed by whoever writes them.

Never commit

- Datasets, volunteer photos, raw videos, or large model files (GitHub limit is 100 MB per file). Link them from the README instead.
- API keys, tokens or passwords. Use environment variables and a `.env.example`.
- Virtual environments, __pycache__, notebook checkpoints or compiled LaTeX build files (keep the final report PDF only).
- Code you cannot explain. If AI wrote it, ask the AI to explain it before you commit.

8. Viva readiness

- You must be able to change logic live. Each member prepares **3 live-change drills** for their own module (example: change the top-k from 3 to 5, add a confidence threshold, switch the SVM kernel).
- Hold a mock viva before each deadline: every member walks through another member's module while the owner asks questions.
- Keep one page of notes per module explaining what it does, why it was designed that way, and what you would change.
- Everyone must understand the whole pipeline end to end (data to prediction to UI), not just their own part.

9. Submission checklist (from the instructions)

- Project topic marked **Approved** in the shared sheet before any work starts.
- GitHub repository URL submitted through GCR, with a README that includes setup instructions.
- Compiled LaTeX PDF report submitted through GCR (the proposal and final report must be written in LaTeX).
- Deployment is excluded for this project. The app runs locally and must start with one command from the README steps.
- No large datasets in the repo; dataset links provided in the README.
- Submit at least **24 hours before the deadline**. No late submissions are accepted for any reason.
- Commit history shows balanced contributions from all three members.
- Every member can explain and modify any part of the code in the viva.

Weekly rhythm: a 20-minute sync at the start of each phase to confirm the contracts in section 5, a mid-phase check on open pull requests, and a short demo of working progress at the end.